FlowMesh

> Reliable, event-driven integration and workflow orchestration for high-volume commerce systems.

FlowMesh is a language-agnostic integration and job-processing platform designed to connect ecommerce systems, payment providers, inventory systems, ERPs, WMS/3PL providers, shipping services, CRM platforms, notification systems, and custom internal services.

The goal is not to build "another RabbitMQ wrapper".

The goal is to provide a reliable execution layer between business systems that can absorb traffic spikes, isolate failures, retry safely, preserve events, prevent duplicate processing, and give engineering/operations teams visibility into every workflow.

1. Problem

Modern commerce systems rarely consist of one application.

A typical order can pass through:

```
Storefront
   ↓
Payment
   ↓
Inventory
   ↓
ERP / OMS
   ↓
Warehouse / 3PL
   ↓
Shipping
   ↓
Customer Notification
```

At small scale, teams often connect these systems with direct API calls, cron jobs, scripts, or application-specific integrations.

As traffic and the number of systems increase, this creates several problems.

Problems FlowMesh targets

Point-to-point integrations become difficult to maintain.

A slow downstream service can block the main application.

External APIs can fail or timeout.

API rate limits can cause requests to be rejected.

Retries can create duplicate orders, payments, emails, or inventory updates.

Traffic spikes can overwhelm downstream systems.

Events can be lost when a service crashes.

Teams have poor visibility into failed workflows.

Failed jobs often require manual investigation.

Different systems expose different APIs and data formats.

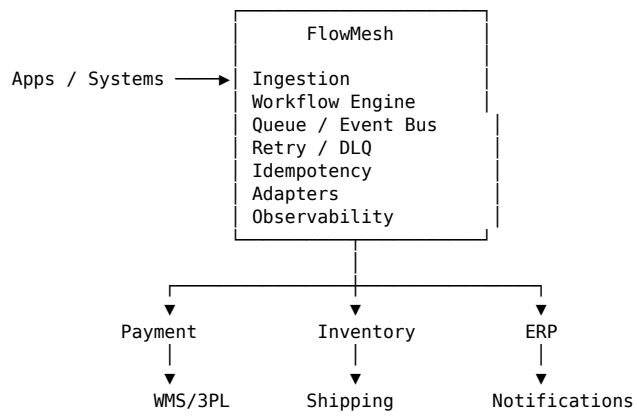
Adding a new provider can require changes to core business code.

There is no central place to inspect retries, failures, latency, and workflow state.

FlowMesh addresses these problems with asynchronous processing, durable job state, idempotency, retries, dead-letter handling, workflow orchestration, adapters, and observability.

2. Product Vision

FlowMesh acts as an integration and workflow layer between business systems.



External applications should not need to know whether FlowMesh internally uses RabbitMQ, Kafka, Redis, or another infrastructure component.

They interact with stable contracts.

3. Target Market

Primary target

E-commerce and commerce operations teams that need reliable integrations between multiple systems.

Examples:

- Growing ecommerce brands
- Retailers
- Marketplaces
- Distributors

- Wholesalers
- Fulfillment companies
- 3PL operators
- SaaS commerce platforms
- Companies with multiple internal services

Why commerce?

Commerce creates workflows where failures have direct business consequences:

- order processing
- payment processing
- inventory synchronization
- fulfillment
- shipment updates
- returns
- refunds
- ERP synchronization
- customer notifications

Peak events such as Black Friday and Cyber Monday make reliability, backpressure, rate-limit handling, retries, and observability especially important.

FlowMesh should therefore be positioned around:

Reliable integration and workflow execution for high-volume commerce.

Black Friday is a major use case, not the entire product.

4. Example Use Case

A customer places an order.

Instead of making the checkout service synchronously call every downstream system:

```
Checkout
 ├── Payment API
 ├── Inventory API
 ├── ERP API
 ├── WMS API
 ├── Shipping API
 └── Email API
```

FlowMesh can orchestrate the process:

```
Checkout
   │
   │ POST /v1/events
   ▼
FlowMesh
   │
```

```
    ├──► Payment Worker
    ├──► Inventory Worker
    ├──► ERP Worker
    ├──► Fulfillment Worker
    └──► Notification Worker
```

If the ERP is temporarily unavailable:

```
ERP Worker
    ↓
Failure
    ↓
Retry with exponential backoff
    ↓
Failure
    ↓
Retry
    ↓
Success
```

If the operation continues to fail:

```
Retry limit reached
        ↓
Dead Letter Queue
        ↓
Operator investigation
        ↓
Replay / Resume
```

The checkout system does not need to remain blocked while the ERP recovers.

5. How Integration Works

FlowMesh should expose a language-agnostic API.

Any application capable of HTTP can integrate.

Event ingestion

```
POST /v1/events
Authorization: Bearer <api-key>
Idempotency-Key: 9b4d0c...

Content-Type: application/json

{
  "type": "order.created",
  "source": "shopify",
  "timestamp": "2026-09-29T12:00:00Z",
  "payload": {
    "orderId": "ORD-10001",
    "customerId": "CUS-100",
    "total": 125.50
  }
}
```

Response:

```
{
  "jobId": "job_01J...",
  "status": "QUEUED"
}
```

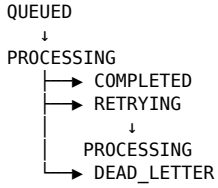
Check job status

```
GET /v1/jobs/:jobId
```

Example:

```
{
  "jobId": "job_01J...",
  "status": "PROCESSING",
  "attempts": 2
}
```

Supported status model



Additional states can include:

- CANCELLED
- PAUSED
- COMPENSATING
- COMPENSATED

6. Integration Methods

FlowMesh should support multiple integration methods over time.

6.1 REST API

For systems that can send HTTP requests.

6.2 Webhooks

Receive events from platforms such as ecommerce, payment, shipping, and SaaS providers.

6.3 Worker SDK

A worker can be implemented in different languages.

Example:

- NestJS Worker
- Go Worker
- Python Worker
- Ruby Worker
- Java Worker

The worker communicates using the FlowMesh contract rather than depending on the implementation language.

6.4 Connector / Adapter

FlowMesh can provide reusable adapters:

- Shopify Adapter
- Stripe Adapter
- SAP Adapter
- NetSuite Adapter
- WooCommerce Adapter
- DHL Adapter
- Custom HTTP Adapter

6.5 Future integration methods

- Kafka
- gRPC
- SFTP / file ingestion
- EDI
- GraphQL
- Scheduled jobs
- Event replay

7. What Can FlowMesh Integrate?

The platform should not be limited to one provider.

Ecommerce

- Shopify
- WooCommerce
- BigCommerce
- Magento
- Custom storefronts
- Marketplaces

Payment

- Stripe
- PayPal
- Adyen
- Local payment gateways
- Custom payment services

ERP

- SAP

- NetSuite
- Microsoft Dynamics
- Odoo
- Custom ERP

Inventory / Warehouse

- WMS
- Inventory services
- 3PL providers
- Warehouse APIs

Shipping

- DHL
- FedEx
- UPS
- Local shipping providers
- Custom carrier APIs

CRM / Customer Support

- Salesforce
- HubSpot
- Zendesk
- Custom CRM

Communication

- Email providers
- SMS providers
- Push notification services
- Slack / internal notifications

Internal systems

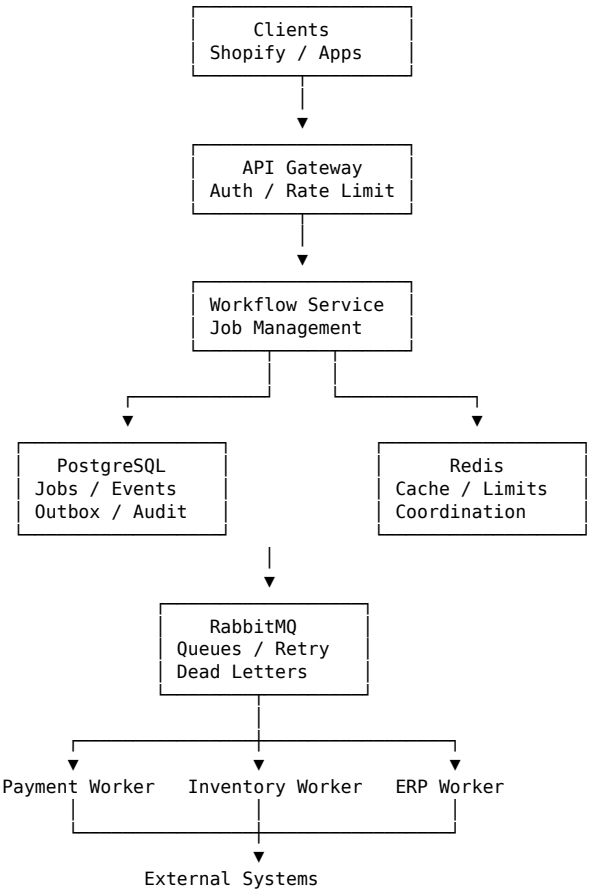
- NestJS
- Node.js
- Go
- Python
- Ruby
- Java
- .NET
- Legacy applications

The key principle is:

If a system exposes a supported protocol, FlowMesh should be able to integrate with it.

8. Architecture

The initial architecture should prioritize reliability and simplicity.



9. Core Components

API Gateway

Responsibilities:

- Authentication
- API keys
- Tenant identification
- Rate limiting
- Request validation
- Idempotency
- Request tracing

Workflow Service

Responsibilities:

- Create jobs
- Create workflow executions
- Track state
- Validate workflow definitions
- Manage dependencies
- Trigger next steps

Message Broker

Initial technology: **RabbitMQ**

Responsibilities:

- Queueing
- Delivery
- Backpressure
- Retry routing
- Dead-letter queues
- Consumer isolation

Later: **Kafka** for high-throughput event streaming and replay-oriented workloads.

PostgreSQL

PostgreSQL is the source of truth for durable business state.

Potential tables:

- tenants
- api_keys
- events
- jobs
- job_attempts
- workflows
- workflow_runs
- workflow_steps
- idempotency_keys
- outbox_events
- dead_letter_jobs
- audit_logs
- connectors
- connector_credentials

Important indexes should be designed around actual access patterns.

Redis

Redis should not become the source of truth for critical business state.

Possible uses:

- Rate limiting
- Short-lived caching
- Distributed coordination where appropriate
- Worker coordination
- Temporary state
- Fast dashboard reads

10. Reliability Model

Reliability is the central engineering goal.

FlowMesh should prefer:

At-least-once delivery + idempotent processing

rather than promising exactly-once execution across arbitrary external systems.

Exactly-once behavior is difficult to guarantee when external APIs, networks, databases, and processes are involved.

10.1 Idempotency

Every externally retriable operation should support an idempotency key.

```
Idempotency-Key: 5f2a...
```

The same logical operation should reuse the same key when retried.
Different business operations should use different keys.

The database can store:

- tenant_id
- idempotency_key
- operation
- request_hash
- status
- response
- created_at
- expires_at

A unique constraint should protect the key:

```
UNIQUE(tenant_id, idempotency_key, operation)
```

10.2 Transactional Outbox

When application state and an outgoing event must be committed together:

```
BEGIN

INSERT order

INSERT outbox_event

COMMIT
```

A worker later publishes the outbox event.

This prevents the classic failure:

```
DB commit succeeds
        ↓
Application crashes
        ↓
Event was never published
```

10.3 Retry

Use controlled retries.

```
Attempt 1
   ↓
1 second
   ↓
Attempt 2
   ↓
5 seconds
   ↓
Attempt 3
   ↓
30 seconds
   ↓
Attempt 4
   ↓
Dead Letter Queue
```

Use exponential backoff with jitter.

Not every error should be retried.

Retryable examples

- HTTP 429
- Temporary network failure
- HTTP 502
- HTTP 503
- Connection timeout

Usually non-retryable examples

- Invalid payload
- Authentication failure
- Unsupported operation

- Permanent business validation error

11. Dead Letter Queue

A DLQ stores jobs that could not be successfully processed after the configured retry policy.

Operators should be able to:

- Inspect the failure
- See all attempts
- View error messages
- View request/response metadata
- Replay the job
- Cancel the job
- Fix the connector
- Resume processing

12. Backpressure

A core requirement for peak traffic.

Suppose:

```
Incoming:
10,000 jobs/sec

ERP capacity:
1,000 jobs/sec
```

The system should not send all 10,000 directly to the ERP.

Instead:

```
Incoming traffic
      ↓
Queue
      ↓
Controlled workers
      ↓
ERP
```

The queue becomes a buffer.

Workers can control concurrency.

This protects downstream systems from overload.

13. Rate Limiting

External providers may impose limits.

Example:

```
Provider:
100 requests/sec
```

FlowMesh can control worker concurrency:

```
Queue
 ↓
Rate limiter
 ↓
Workers
 ↓
External API
```

The system should support per:

- Tenant
- Connector
- Provider
- Endpoint
- API key

14. Circuit Breaker

If a downstream provider is failing continuously:

```
Healthy
   ↓
Failures increase
   ↓
OPEN
   ↓
Stop sending requests temporarily
   ↓
Cooldown
   ↓
HALF OPEN
   ↓
Test request
   ↓
Healthy → CLOSED
Failure → OPEN
```

This prevents cascading failures.

15. Workflow / Saga

Complex commerce operations may contain multiple steps.

Example:

```
Create Order
     ↓
Reserve Inventory
     ↓
Charge Payment
     ↓
Create Fulfillment
     ↓
Send Notification
```

If a later step fails, the workflow may need compensation.

Example:

```
Reserve Inventory
       ↓
Payment
       ↓
Payment fails
       ↓
Release Inventory
```

This is where the **Saga pattern** becomes useful.

FlowMesh should model workflows as state machines / directed graphs.

16. Design Patterns to Focus On

16.1 Adapter Pattern

Different providers expose different APIs.

```
PaymentProvider
      │
      ├── StripeAdapter
      ├── PayPalAdapter
      └── AdyenAdapter
```

The workflow should depend on an interface, not a provider implementation.

16.2 Strategy Pattern

Useful when behavior changes based on configuration.

```
RetryStrategy
 ├── FixedBackoff
 ├── ExponentialBackoff
 └── ExponentialBackoffWithJitter
```

16.3 Factory Pattern

Useful for selecting connectors dynamically.

```
ConnectorFactory
      ↓
Stripe
Shopify
SAP
DHL
...
```

16.4 Observer / Event-Driven Pattern

Useful when multiple consumers react to an event.

```
order.created
      │
      ├── Inventory
```

```
        ├── Analytics
        ├── Notification
        └── ERP
```

16.5 State Machine

Useful for job and workflow lifecycle management.

```
QUEUED
  ↓
PROCESSING
  ↓
COMPLETED
```

or:

```
PROCESSING
  ↓
FAILED
  ↓
RETRYING
  ↓
PROCESSING
```

16.6 Saga Pattern

For distributed workflows requiring compensation.

16.7 Transactional Outbox

For reliable database-to-message publication.

17. Architecture Principles

FlowMesh should follow these principles.

- **Loose Coupling** — Services should communicate through stable contracts instead of direct dependencies.
- **Durability** — Important events and job state must survive process crashes.
- **Idempotency** — Retries must not accidentally create duplicate business operations.
- **Backpressure** — Slow downstream systems must not bring the whole platform down.
- **Observability** — Every important operation should be traceable.
- **Horizontal Scalability** — Workers and API services should scale independently.
- **Failure Isolation** — One connector should not take down unrelated workflows.
- **Contract First** — Events and APIs should have explicit schemas and versioning.
- **Security by Default** — Secrets, credentials, and tenant data must be protected.
- **Operational Recovery** — Failed operations should be inspectable and replayable.
- **Multi-Tenancy** — Tenant data and execution must be isolated.
- **Source of Truth** — PostgreSQL owns durable job/workflow state.

18. Multi-Tenancy

FlowMesh should be designed as a multi-tenant platform from the beginning.

Every important resource should contain:

```
tenant_id
```

Example:

```
jobs
---------
id
tenant_id
workflow_id
status
created_at
```

Isolation should exist at multiple layers:

```
Authentication
      ↓
Tenant Context
      ↓
Authorization
      ↓
Database Isolation
      ↓
Queue Isolation
```

Possible future strategies:

- Shared database / shared schema
- Shared database / separate schema
- Separate database per tenant

The initial version can start with shared schema + strict tenant scoping and evolve as requirements grow.

19. Observability

A distributed system without observability becomes difficult to operate.

FlowMesh should expose:

Metrics

- Jobs/sec
- Queue depth
- Processing latency
- p50 / p95 / p99
- Error rate
- Retry rate
- DLQ size

- Worker utilization
- External API latency
- Rate-limit events

Logs

Every job should have correlation information.

```
trace_id
job_id
workflow_id
tenant_id
attempt_id
connector
```

Distributed Tracing

Use **OpenTelemetry**.

```
API Request
   ↓
Create Job
   ↓
Publish Message
   ↓
Worker
   ↓
External API
```

The complete path should be traceable.

20. Security

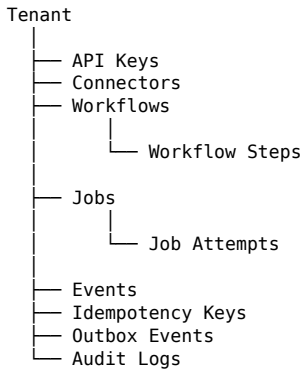
Security requirements:

- API keys
- OAuth where needed
- Secret encryption
- Tenant isolation
- Role-based access control
- Audit logs
- Request validation
- Rate limiting
- TLS
- Credential rotation
- Least privilege
- Webhook signature verification

Connector credentials should never be stored as plaintext.

21. Data Model

Initial conceptual model:



22. Suggested Tech Stack

Backend

- Node.js
- TypeScript
- NestJS

Database

- PostgreSQL

ORM

- Prisma
- TypeORM

For the first implementation, Prisma or TypeORM is sufficient. The architecture should not depend heavily on the ORM.

Messaging

- Initial: RabbitMQ
- Future: Kafka

Cache / Coordination

- Redis

Infrastructure

- Docker
- Docker Compose
- Future: Kubernetes
- Future: AWS

Observability

- OpenTelemetry
- Prometheus
- Grafana

API Documentation

- OpenAPI / Swagger

23. MVP

The first version should **NOT** attempt to integrate everything.

Build a strong vertical slice.

Phase 1

Authentication

- Tenant creation
- API key generation
- API authentication

Event ingestion

- `POST /v1/events`
- `Idempotency-Key`

Job management

- `GET /v1/jobs/:id`

Queue

- RabbitMQ

Worker

- Build one worker that executes a real external integration.

Retry

- Exponential backoff + jitter

DLQ

- Store failed jobs

Dashboard

- Jobs
- Status
- Attempts
- Errors
- Queue depth

- DLQ

Replay

- Replay failed jobs

24. First Real Workflow

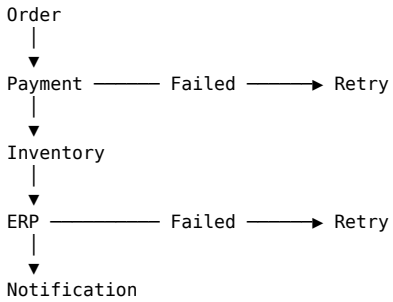
A good first workflow is:

```
order.created
      ↓
Payment
      ↓
Inventory
      ↓
ERP
      ↓
Notification
```

Each step should have:

- timeout
- retry policy
- idempotency
- status
- attempts
- logs
- trace

Example:



25. Example Failure Scenario

Imagine Black Friday.

```
Normal:
500 orders/min

Peak:
10,000 orders/min

ERP capacity:
1,000 orders/min
```

Without FlowMesh:

```
Checkout
   ↓
ERP
   ↓
Timeout
   ↓
Checkout errors
```

With FlowMesh:

```
Checkout
   ↓
FlowMesh
   ↓
Queue
   ↓
Workers
   ↓
ERP
```

The queue absorbs the temporary difference between incoming traffic and downstream processing capacity.

This is the kind of problem the platform should be designed to solve.

26. Competitors / Existing Market

FlowMesh is entering an existing integration/automation category.

Examples include:

- Celigo
- Patchworks
- Workato
- Boomi
- MuleSoft
- Zapier

These platforms demonstrate that businesses already pay for integration, automation, orchestration, monitoring, and connector ecosystems.

The goal of FlowMesh is not to copy these products.

The project should use a narrower technical positioning:

Developer-first, reliability-focused integration and workflow infrastructure for high-volume commerce.

The differentiating engineering story should be:

- Durable asynchronous execution
- Strong idempotency model
- Retry and DLQ
- Backpressure

- Rate-limit awareness
- Circuit breakers
- Workflow orchestration
- Replay
- Observability
- Multi-tenancy
- Language-agnostic workers

27. Competitive Positioning

A useful positioning matrix:

| Product Category | Main Focus |

|---|---|

| Workflow automation | Easy business automation |

| iPaaS | Connecting business applications |

| Message broker | Transporting messages |

| Job queue | Background processing |

| Workflow engine | Long-running workflows |

| **FlowMesh** | **Reliable commerce integration + workflow execution** |

FlowMesh should sit between these categories.

It is not simply:

- RabbitMQ UI
- Zapier clone

It should be an infrastructure product for developers and engineering teams.

28. API Contract Philosophy

The public API should remain stable even when internal infrastructure changes.

```
Client
  ↓
FlowMesh API
  ↓
RabbitMQ
```

Later:

```
Client
  ↓
FlowMesh API
  ↓
Kafka
```

The client should not need to change.

This abstraction is important.

29. Event Versioning

Events should be versioned.

```
order.created.v1
order.created.v2
```

A schema should define:

- Required fields
- Optional fields
- Data types
- Version
- Compatibility rules

Future technologies:

- JSON Schema
- Avro
- Protobuf

30. Workflow Versioning

A workflow can change over time.

```
Order Workflow v1

Payment
  ↓
Inventory
  ↓
ERP
```

Later:

```
Order Workflow v2

Fraud Check
  ↓
Payment
  ↓
Inventory
  ↓
ERP
  ↓
Notification
```

Existing executions should continue using the workflow version they started with.

New executions use the new version.

31. Developer Experience

A major part of the product should be the developer experience.

A developer should be able to:

- Create tenant
- Generate API key
- Configure connector
- Define workflow
- Send event
- Watch execution
- Inspect failure
- Replay job

Example:

```
curl -X POST https://api.flowmesh.dev/v1/events \
  -H "Authorization: Bearer $FLOWMESH_API_KEY" \
  -H "Idempotency-Key: order-10001" \
  -H "Content-Type: application/json" \
  -d '{
    "type": "order.created",
    "source": "my-store",
    "payload": {
      "orderId": "10001"
    }
  }'
```

32. Project Structure

A possible NestJS monorepo:

```
flowmesh/
├── apps/
│   ├── api/
│   ├── worker/
│   └── dashboard/
│
├── libs/
│   ├── auth/
│   ├── database/
│   ├── events/
│   ├── messaging/
│   ├── workflows/
│   ├── idempotency/
│   ├── retry/
│   ├── connectors/
│   ├── observability/
│   └── common/
│
├── infrastructure/
│   ├── docker/
│   ├── rabbitmq/
│   ├── postgres/
│   └── redis/
│
├── docs/
│   ├── architecture/
│   ├── api/
│   └── decisions/
│
└── docker-compose.yml
```

33. Engineering Decision Records

Important architectural decisions should be documented.

Examples:

- ADR-001: Use PostgreSQL as durable source of truth
- ADR-002: Use RabbitMQ for initial messaging
- ADR-003: Use at-least-once delivery
- ADR-004: Require idempotent consumers
- ADR-005: Use Transactional Outbox
- ADR-006: Use exponential backoff with jitter
- ADR-007: Use OpenTelemetry
- ADR-008: Start with shared-schema multi-tenancy

This makes the project look like an engineering system rather than a tutorial project.

34. What NOT to Build Initially

Avoid building:

- 100 connectors
- Kubernetes deployment
- Multi-region architecture
- Kafka + RabbitMQ simultaneously
- Complex visual workflow builder
- AI agents
- Full EDI engine
- Enterprise billing
- Massive dashboard
- Exactly-once guarantees

First prove:

```
Event
 ↓
Durable Job
 ↓
Queue
 ↓
Worker
 ↓
External API
 ↓
Retry
 ↓
DLQ
 ↓
Replay
```

If this works reliably, the foundation is valuable.

35. Long-Term Roadmap

V1

- REST API
- API keys
- Multi-tenancy
- PostgreSQL
- RabbitMQ
- Redis
- Idempotency
- Retry
- DLQ
- One or two connectors
- Basic dashboard
- OpenTelemetry

V2

- Workflow builder
- Connector SDK
- More integrations
- Workflow versioning
- Scheduled jobs
- Priority queues
- Rate-limit policies
- Replay

V3

- Kafka support
- Event streaming
- Event replay
- Schema registry
- Advanced workflow orchestration
- gRPC
- More enterprise integrations

V4

- Kubernetes

- Multi-region
- Advanced tenant isolation
- Connector marketplace
- Billing
- Enterprise governance

36. Success Metrics

Technical metrics

- Job success rate
- p95/p99 processing latency
- Retry rate
- DLQ rate
- Queue depth
- Worker throughput
- Connector error rate
- External API error rate
- Event loss rate
- Replay success rate

Business/product metrics

- Active tenants
- Workflows executed
- Events processed
- Connected systems
- Connector usage
- Integration setup time

37. Core Principle

The most important concept behind FlowMesh is:

> **Do not make every business system responsible for reliability. Put reliability into the integration layer.**

The application should be able to say:

> "Here is an order event."

FlowMesh should take responsibility for:

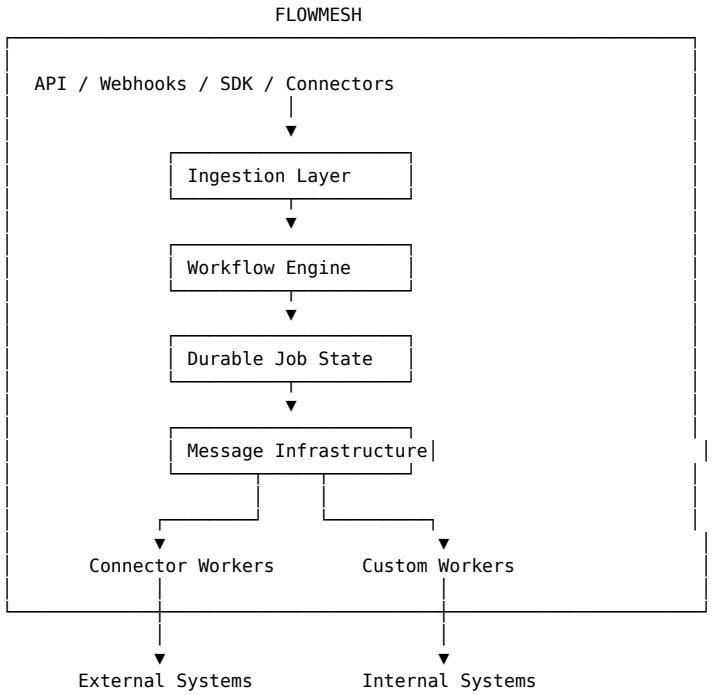
- durability

- routing
- execution
- retry
- backpressure
- rate limits
- idempotency
- failure isolation
- observability
- replay

That is the core product.

38. Final Architecture Goal

The long-term architecture should evolve toward:



FlowMesh should ultimately become a reliable execution layer between applications, not merely a queue.

39. Recommended First Implementation

Build the project in this order:

- NestJS API
- PostgreSQL schema
- Tenant + API key authentication

- Event ingestion
- Idempotency
- Transactional Outbox
- RabbitMQ
- Worker service
- Retry engine
- DLQ
- Job attempts
- Replay
- OpenTelemetry
- Prometheus/Grafana
- First real connector
- End-to-end commerce workflow
- Dashboard
- Load testing

Then test it under simulated peak traffic.

A meaningful demonstration would be:

```
10,000 incoming events
        ↓
FlowMesh
        ↓
Queue
        ↓
Controlled workers
        ↓
Artificial downstream failure
        ↓
Retries
        ↓
Recovery
        ↓
Completion
```

The demo should show that FlowMesh can absorb spikes, survive downstream failures, avoid duplicate processing, and provide complete visibility into what happened.

40. Summary

FlowMesh is a developer-first integration and workflow platform for high-volume commerce.

It connects:

Ecommerce

Payment

Inventory

ERP

WMS

3PL

Shipping

CRM

Notifications

Internal Services

through reliable asynchronous workflows.

Its core engineering concepts are:

- Event-driven architecture
- Message queues
- Transactional Outbox
- Idempotency
- Retry + Backoff
- Dead Letter Queue
- Backpressure
- Rate Limiting
- Circuit Breaker
- Saga
- Adapters
- Workflow State Machines
- Multi-tenancy
- Observability
- Horizontal Scalability

The project should be built as a serious distributed-systems portfolio project with production-oriented engineering decisions, failure scenarios, load testing, documentation, and operational visibility.

FlowMesh — Reliable workflows between the systems that run commerce.